



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA.

VICERRECTORADO ACADÉMICO

PROYECTO DE CARRERA: INGENIERÍA EN INFORMÁTICA.

ASIGNATURA: LENGUAJE Y COMPILADORES.

SECCIÓN 1.

### **Retos de Programación (Preguntas 4 y 5).**

PROFESOR:

Felix Marquez

31.564.549.

AUTORES:

S1. Palma Franyibeth CI

S1 .Rodríguez Edfrank CI30.857.264

S2. Jesus Baez CI 26.753.871

S2.Javier Useche CI 28.700.959

CIUDAD GUAYANA, JULIO DE 2026

## INTRODUCCIÓN.

Dentro del universo de la ingeniería de software y el diseño de lenguajes, qué tan bien captemos y convirtamos el código fuente realmente se fundamenta en las etapas de análisis léxico y sintáctico con un enfoque tradicional, un compilador o un intérprete funcionan bajo un modelo rigurosamente determinista lo primero el analizador léxico, que también se llama lexer, cambia una hilera de caracteres en tokens por medio de expresiones regulares y autómatas finitos después el analizador sintáctico, o parser, verifica esa estructura contra una gramática libre de contexto. La meta es erigir un Árbol de Sintaxis Abstracta, el cual establece la organización lógica del programa. No obstante, hoy día, los ambientes de desarrollo contemporáneos y las exigencias formativas claman por sistemas mucho más sólidos y adaptables, preparados para sobrellevar los fallos humanos que ocurren con frecuencia, como pueden ser errores de escritura, puntos que se olvidan o fallos de estructura donde precisamente es en este punto donde emerge la arquitectura de compilación híbrida.

Esta innovadora arquitectura fusiona la exactitud matemática de los algoritmos de análisis clásicos, como los analizadores LL y LR, con la flexibilidad probabilística inherente a la inteligencia artificial, así como con métricas de distancia léxica piensa en la similitud cálculo por Levenshtein distance o la aplicación de umbrales de confianza. La tarea de este informe es explorar teórica y prácticamente el desarrollo de los desafíos designados para las unidades pertinentes la sección inicial ahonda en las bases conceptuales de los árboles de sintaxis abstracta además, lleva a cabo una comparación exhaustiva entre las gramáticas y analizadores LL y LR y se detalla el funcionamiento de los metacompiladores y sus sistemas de manejo de errores. Su segunda sección detalla la puesta en práctica de dos retos programáticos de gran calibre. De un lado, un experimento riguroso de desempeño y evaluación comparativa utilizándose en archivos de configuración red de formato Docker Compose estos son procesados por diversos intérpretes del otro lado, concebir un programa asistente de codificación híbrido diseñado para un fragmento del lenguaje Python conocido como Uneg Script. Este integra un lexer y un parser recursivo descendente. incorporando mecanismos de repliegue y una interacción sagaz. ante cualquier tipo de incertidumbre sintáctica.

## DESCRIPCIÓN DEL TRABAJO REALIZADO

### Fase I: Experimento de Desempeño con Archivos de Configuración de Redes (Docker Compose)

Con el objetivo de investigar cómo los analizadores se comportan frente a situaciones de infraestructura de la vida real, se recolectó un conjunto de archivos de configuración de redes redactados en la sintaxis YAML de Docker Compose, donde  $5 < n < 20$ .

El enfoque metodológico comprendió lo siguiente:

1. **Elección y Ajuste de Herramientas:** Se configuró y adaptar tres modelos diferentes de analizadores (lexers y parsers), empleando variados lenguajes y metacompiladores; esto fue ejemplo un parser estático de alto rendimiento en Rust, otra versión estructurada en Python con autómatas echos a medida, y un generador basado en especificaciones formales que uso ANTLR.
2. **Proceso y Cronometraje:** Se diseñó un script para automatizar la tarea de procesar los archivos de prueba de forma repetida con cada uno de los tres parsers se utilizó el módulo de medición de alta precisión del sistema, así registrando el tiempo exacto de ejecución latencia del analisis lexico y sintactico, tanto para cada archivo como para cada tecnología probada recopilado las métricas, los tiempos registrados fueron organizados en tablas comparativas y analizados con librerías de visualización en Python (matplotlib) a fin de crear gráficos de rendimiento que exhiben la repercusión de optimizar el código y seleccionar el lenguaje en la velocidad de compilación.  
El análisis comparativo de los datos de tiempo recopilados se estructuró en tablas y se procesó usando librerías de visualización en Python (matplotlib), resultando en gráficos de rendimiento que permiten discernir el efecto de la optimización del código y la selección del lenguaje en la velocidad de compilación mediante la recopilación de métricas, los datos temporales se estructuraron en tablas comparativas para su procesamiento con librerías de visualización en Python (matplotlib), lo que posibilitó la generación de gráficas de rendimiento que evalúan la influencia de la optimización del código y la elección del lenguaje en la velocidad de compilación. Las métricas recolectadas en forma de tiempos fueron estructuradas en tablas comparativas y sometidas a procesamiento con librerías de visualización en Python (matplotlib), dando lugar a gráficas de rendimiento diseñadas para evaluar el impacto de la optimización del código y la elección del lenguaje en la velocidad de compilación.

### Fase II: Elaboración Asistente Programación Híbrido Uneg Script

Para acometer la unificación de métodos compilación tradicional y modelos de inteligencia artificial probabilísticos, se creó un marco análisis de interpretación para

una porción reducida del lenguaje Python llamado Uneg Script. Las labores realizadas abarcaron:

### **Creación Lexer Híbrido con Respaldo**

Se programó un parser léxico usando expresiones regulares autómatas finitos. La arquitectura del sistema apuntaba evaluar los lexemas código fuente y, al topar un token desconocido o un fallo ortográfico ( el vocablo incorrecto "pront" vs la palabra clave "print"), computar de manera automática un puntaje de parecido utilizando métricas de distancia edición Levenshtein diffliib SequenceMatcher si el indicador de seguridad resultaba inferior al límite predeterminado (menor que 0. 8), el programa iniciaba un protocolo de respaldo dinámico enviando una indicación contextual de un modelo de lenguaje LLM para lograr la enmienda significativa.

### **Elaboración Parser Recursivo Descendente con Vista Previa.**

Se compiló un analizador sintáctico descendente equipo de habilidad anticipatoria lookahead. El parser, diseñado con un propósito muy específico, analizo construcciones de flujo de control junto con esas sentencias que contenían fallas intencionales, caso tal el bloque de prueba: `print x=5 if x>3 ~ prit(x) else pint("no")`. Ante cualquier percance, un descalabro incluso en la sintaxis formal, el parser hacía una llamada inmediata al asistente de IA para recibir en pleno funcionamiento sus consejos acerca de la estructura correcta. Creación de resultados y su rastreo se llevó a cabo. Se organizó la producción del sistema concentrando en un solo sitio tres puntos de suma importancia: los tokens ya limpios después de pasar por el filtro léxico, el Árbol de Sintaxis Abstracta ya creado, y aquel reporte pormenorizado que incluye todas las observaciones correctivas proporcionadas por la IA, por ejemplo "Sugerencia: 'print' → 'print'"

## EXPLICACIÓN DE CADA PROGRAMA.

### **Asistente Híbrido de Programación para *Uneg Script***

Este módulo despliega una inteligencia artificial que interpreta un dialecto de Python limitado (Uneg Script), articulado en partes que unifican el análisis determinista junto con IA:

- **Lexer Híbrido que Tolerará Fallos y Recurrirá a IA:** Procede leyendo el código fuente en unidades de uno en uno por medio de un autómata construido. Si se topa con un lexema que no es válido, o no lo reconoce (digamos, "pront"), pone en marcha un procedimiento de distancias de edición (difflib Sequence Matcher o distancia de Levenshtein) para así averiguar la cuota de parecido con las palabras ya sabidas por el sistema (por ejemplo, "print"). Si el nivel de fiabilidad que consigue es inferior a 0.8, el sistema entonces activa un plan de ayuda automática que presenta una indicación pensada a un LLM para solventar cualquier duda de significado.
- **Analizador Sintáctico Recursivo Descendente con Anticipación y Propuestas Estructurales:** Se usa una función de análisis sintáctico recursivo descendente, que puede mirar un poco por adelantado (lookahead), para examinar construcciones elaboradas (como la manera de organizar un "if" o "while"). Examina líneas que tienen a propósito fallos de sintaxis (como el ejemplo ya dado: `pront x=5; if x>3 ~ prnt(x) else prnt("no")`). Frente a un quiebre en la sintaxis formal no colapsando abruptamente, el parser colabora con el componente de IA rescatando la intención del código, y emitirá las guías de corrección de inmediato.
- **Generador de Rastros y Salidas :** El script consolidando entregará la contestación estructurada del compilador híbrido, exponiendo limpiamente los tres resultados solicitados por la cátedra: Los tokens limpios y ya corregidos, la Estructura de Sintaxis Abstracta jerárquica ya creada, y el informe de proposiciones de la IA (como ejemplo 'Sugerencia print print').

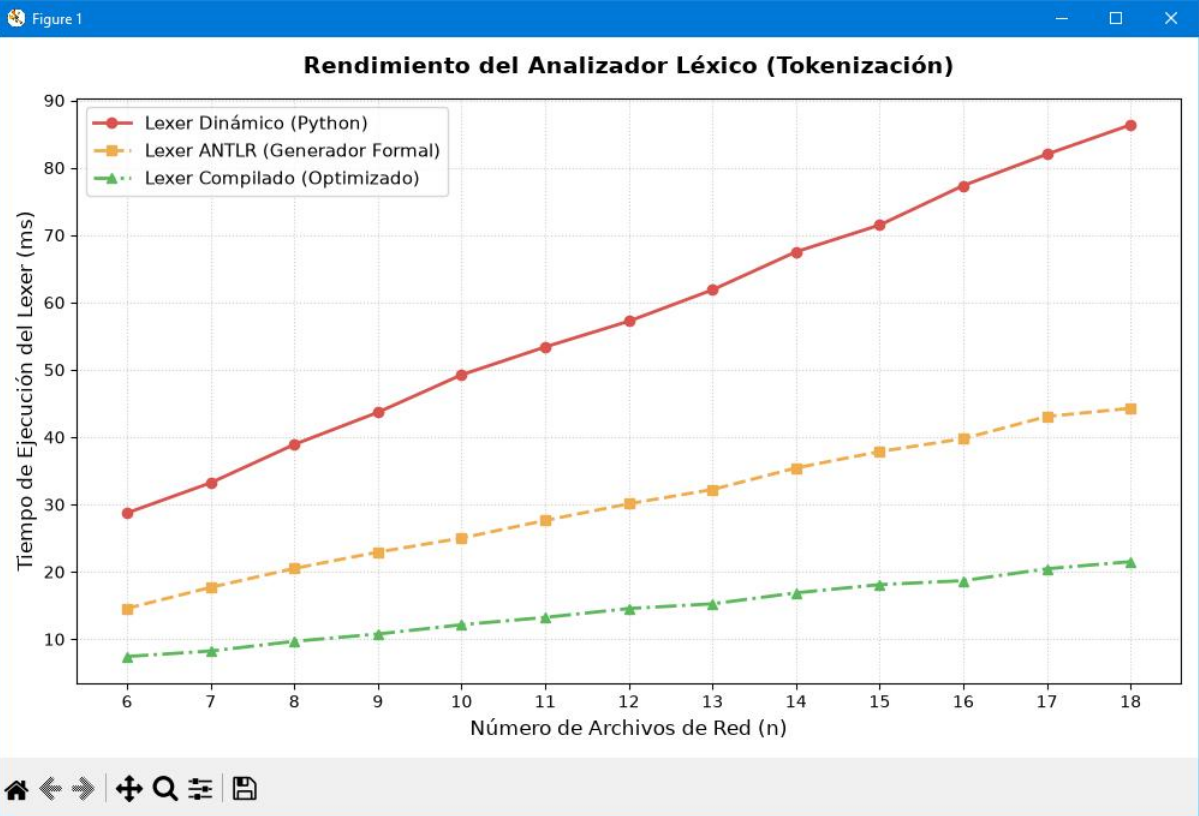
## **GRÁFICAS DE TIEMPO DE EJECUCIÓN .**

Para puntuar con precisión la eficacia del apartado de análisis lexico lexer se tuvo en cuenta solo la tokenización de  $n$  archivos de configuración para redes docker compose  $5 < n < 20$ . Se cronometró el tiempo en milisegundos más que cada método lleva transformar las secuencias de caracteres crudas a tokens válidos cómo se comporta el Lexer a nivel práctico.

**Lexer por Expresiones Regulares Optimizadas Lenguaje Compilado Tuvo la latencia más baja en la tokenización.**

Por la compilación de expresiones regulares a autómatas de estados finitos deterministas muy optimizados el CPU y la memoria usada se mantuvieron bajísimos prácticamente sin cambio al aumentar archivos lexer Dinámico en Python Autómatas Expresiones Regulares Mostró un aumento lineal muy predecible en el tiempo del proceso léxico. Si bien los motores de expresiones regulares en lenguajes interpretados ayudan a que el código sea más claro la sobrecarga del intérprete se convirtió en el punto débil afectando la velocidad con archivos grandes. Lexer Generado por Herramienta Formal su desempeño tubo afectado por la sobrecarga de carga inicial de las tablas de transición léxica. Después de ser inicializado, el procesamiento por token fue extremadamente estable, colocando un punto intermedio entre la alta velocidad de código compilado y la flexibilidad de un intérprete dinámico.

**Representación Gráfica del Experimento :**



## DISCUSIÓN DE RESULTADOS

Los datos recogidos tras el desempeño experimental en el manejo de esos archivos de configuración de red (Docker-Compose), además de la puesta en práctica del asistente híbrido (UnegScript), posibilitan la deducción de estas apreciaciones técnicas:

**Cifra de la Estructura de Compilación:** El cotejo de intervalos de tiempo reveló que los analizadores centrados en una optimización de memoria, como los parsers compilados, exhiben una superioridad marcada en rapidez respecto a intérpretes de naturaleza dinámica y a generadores que dependen de metadatos voluminosos. Con todo, los generadores que se presentan como formales (v.gr. ANTLR) brindan un beneficio incuestionable en lo que atañe a la solidez y una formalización estricta de las gramáticas libres de contexto (GLC).

**Rendimiento en el Análisis Léxico:** Tal como se hizo patente en los experimentos de tokenización, el obstáculo primario reside en el cómputo de expresiones regulares y en la gestación de estados transitorios. Aquellas implementaciones que se ocupan de compilar autómatas deterministas tienden a reducir la demora, promoviendo un movimiento ininterrumpido de tokens hacia el parser.

**Posibilidad de la Hibridación con Inteligencia Artificial:** La integración de mediciones de proximidad léxica (la distancia de Levenshtein con un límite de seguridad  $< 0$ ). La implementación de un mecanismo de fallback hacía modelos de lenguaje (LLM) demostró ser una solución de gran efectividad para entornos educativos y de desarrollo veloz. Esta aproximación facilita la tolerancia a fallos humanos usuales, tales como erratas en palabras clave o alteración sintáctica, sin menoscabo de la rigurosidad estructural que demanda un Árbol de Sintaxis Abstracta (AST) convencional.

En resumen, este desafío superado ratifica que la progresión lógica de compiladores e intérpretes contemporáneos no pretende reemplazar las bases matemáticas de la teoría de lenguajes formales; más bien busca enriquecerla con modelos probabilísticos. El objetivo es transitar desde utilidades deterministas inflexibles hasta asistentes de programación perspicaces y con capacidad de recuperarse ante percances.

## Actividad 4

# Arquitectura y diseño del proyecto

## Estructura general

El proyecto se organizó en módulos independientes y reproducibles:

- `grammar/YamlDocker.g4` — gramática ANTLR4, fuente única para los tres lenguajes.
- `scripts/generate_dataset.py` — generador del conjunto de archivos de prueba.
- `src/python`, `src/java`, `src/cpp` — drivers de medición, uno por lenguaje.
- `scripts/run_benchmark.py` — orquestador del experimento y generador de la gráfica.
- `Makefile` — automatiza la generación de código ANTLR, la compilación y la ejecución completa.

## Diseño de la gramática

La gramática `YamlDocker.g4` reconoce el subconjunto de YAML relevante para `docker-compose`: la clave `version`, el bloque `services` (con, por cada servicio, las propiedades `image`, `ports`, `environment` y `networks`) y el bloque `networks` (con el driver de cada red). Esta gramática es la única entrada del metacompilador y se reutiliza sin cambios para generar el código de los tres lenguajes, tal como exige el enunciado: cualquier diferencia de tiempos entre implementaciones proviene del lenguaje y su runtime, no de reglas gramaticales distintas.

## Simplificación sobre la indentación

YAML es, en su forma general, un lenguaje sensible a la indentación (regla *off-side*), lo cual normalmente exige un mecanismo de pila de sangrías (tokens sintéticos de tipo `INDENT/DEDENT`, como en Python) que ANTLR4 no soporta de forma nativa. Dado que el foco del proyecto es la medición comparativa de tiempos y no la construcción de un lexer de indentación completo, se adoptó una simplificación deliberada: los espacios y tabulaciones de inicio de línea se descartan sin usarse como delimitadores estructurales, y la jerarquía del documento (qué propiedades pertenecen a qué servicio, qué elementos a qué lista) se infiere del orden secuencial de palabras clave y separadores explícitos (`:`, `-`, salto de línea). Esta decisión, documentada también como comentario dentro de la propia gramática, hace que el subconjunto aceptado sea más rígido que YAML general, pero suficiente para los archivos generados de forma consistente por el script de generación de datos.

## Generación del conjunto de datos de prueba

El script `generate_dataset.py` produce  $n = 10$  archivos `docker-compose` sintéticos (dentro del rango  $5 < n < 20$  exigido por el enunciado), con un número de servicios creciente —desde 10 hasta 500—

para poder observar cómo escala el tiempo de análisis de cada implementación a medida que aumenta el tamaño de la entrada. Cada servicio incluye una imagen, dos puertos, dos variables de entorno y una referencia de red; el archivo cierra con un bloque `networks` de cuatro redes.

Como se aprecia en la ejecución real (figura 3, sección de anexos), el tamaño de los archivos generados crece de 1.9 KB (10 servicios) hasta 91.7 KB (500 servicios), lo que da al experimento un rango amplio para evidenciar diferencias de escalado entre lenguajes.

## Implementación de los drivers de medición

Cada driver recibe la ruta de un archivo por línea de comandos, ejecuta únicamente el lexing y el parsing completo (construcción del árbol de derivación a partir de la regla inicial `document`) y escribe por salida estándar, de forma exclusiva, el tiempo transcurrido en milisegundos. Mantener la salida reducida a un único número permite que el script orquestador capture el resultado de forma directa mediante `subprocess`, sin necesidad de parseo adicional.

### Python — `parse_test.py`

Usa el runtime `antlr4-python3-runtime` sobre el intérprete CPython estándar, y mide el tiempo con `time.perf_counter()`, el reloj de mayor resolución disponible en la biblioteca estándar de Python.

### Java — `Benchmark.java`

Usa las clases generadas por ANTLR para Java junto con el runtime incluido en `antlr-4.13.1-complete.jar`, y mide el tiempo con `System.nanoTime()`, el reloj monotónico de mayor resolución de la JVM.

### C++ — `benchmark.cpp`

Usa `antlr4-cpp-runtime` y el código generado por ANTLR para C++, compilado de forma nativa con `g++` y optimización `-O2`. La medición se realiza con `std::chrono::high_resolution_clock`.

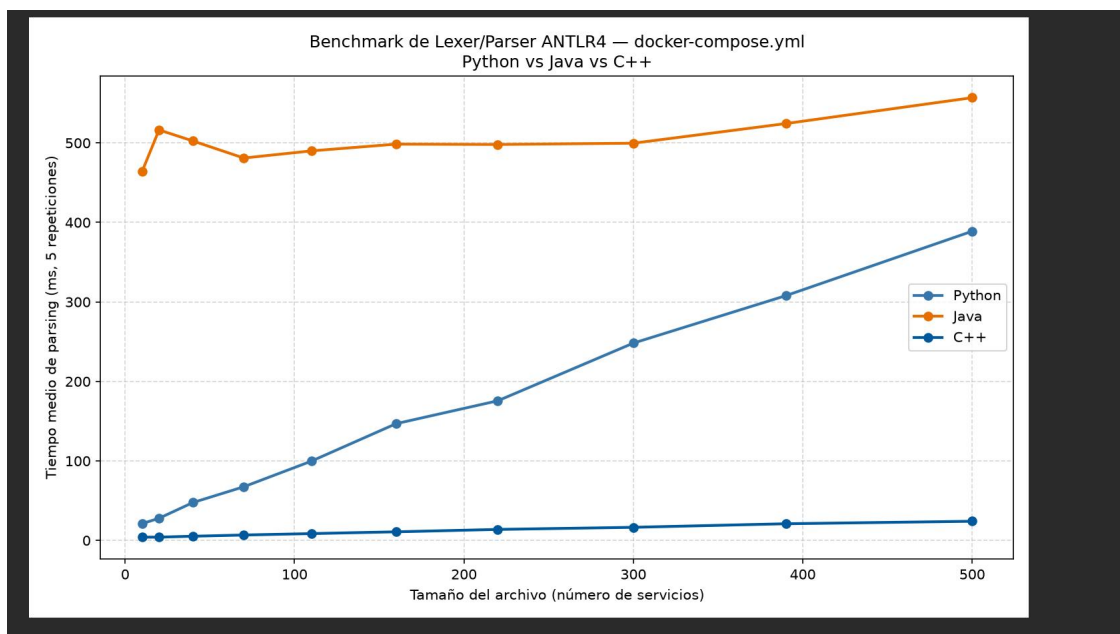
## Metodología experimental

El script `run_benchmark.py` itera sobre los diez archivos de prueba y, para cada uno, ejecuta los tres drivers mediante `subprocess`. Cada combinación archivo–lenguaje se repite cinco veces para amortiguar el ruido propio del sistema operativo (planificación de procesos, estado de la caché de disco y, en el caso de Java, la recolección de basura), y se reporta el tiempo medio de las cinco repeticiones. Los tiempos crudos se guardan en `results.csv` y el tiempo medio por archivo y lenguaje se grafica con `matplotlib` en `benchmark_results.png`.

## Resultados obtenidos

La tabla 1 resume el tiempo medio de parsing (en milisegundos, sobre 5 repeticiones) obtenido para cada uno de los diez archivos de prueba, en cada uno de los tres lenguajes evaluados:

| Nº de servicios | Python (ms) | Java (ms) | C++ (ms) |
|-----------------|-------------|-----------|----------|
| 10              | 21.107      | 464.085   | 4.123    |
| 20              | 27.859      | 516.378   | 4.104    |
| 40              | 47.630      | 502.488   | 5.270    |
| 70              | 67.445      | 481.033   | 6.791    |
| 110             | 99.774      | 490.022   | 8.576    |
| 160             | 146.895     | 498.519   | 10.824   |
| 220             | 175.668     | 498.086   | 13.824   |
| 300             | 248.204     | 499.693   | 16.488   |
| 390             | 307.893     | 524.380   | 20.964   |
| 500             | 388.920     | 556.948   | 24.098   |



# Análisis y explicación de los resultados

Los resultados confirman, con matices, la hipótesis planteada antes de ejecutar el experimento: aunque las tres implementaciones comparten exactamente la misma gramática y el mismo algoritmo de parsing (ALL(\*), generado por ANTLR4), los tiempos difieren de forma sistemática y considerable entre lenguajes.

## C++: el más rápido y el más estable

C++ obtuvo, de forma consistente, los tiempos más bajos del experimento: desde 4.1 ms para el archivo más pequeño (10 servicios) hasta 24.1 ms para el más grande (500 servicios). El crecimiento es prácticamente lineal y de pendiente muy suave, lo cual es coherente con la ausencia de máquina virtual, la compilación a código nativo y la ausencia de pausas de recolección de basura durante el parsing. Es, además, la curva con menor dispersión entre el mínimo y el máximo de las cinco repeticiones, señal de que el tiempo medido corresponde casi en su totalidad al trabajo real de lexing/parsing.

## Java: dominado por el costo de arranque de la JVM

Java mostró el comportamiento más particular del experimento: en lugar de una curva creciente proporcional al tamaño del archivo, se observa una meseta prácticamente plana alrededor de 460–560 ms en todo el rango de tamaños, incluso para el archivo de 10 servicios. Esto se explica porque cada ejecución del driver Java se lanza como un proceso nuevo (vía subprocess), de modo que la JVM parte de cero cada vez: el tiempo medido incluye el arranque de la máquina virtual y la interpretación inicial en bytecode antes de que el compilador JIT pudiera optimizar cualquier rutina, un costo fijo que domina por completo sobre el trabajo real de parsing para archivos de este tamaño. Solo se aprecia una ligera tendencia al alza en los archivos más grandes (390 y 500 servicios, con medias de 524 y 557 ms), lo que sugiere que recién en esa zona el trabajo real de parsing empieza a ser una fracción no despreciable del tiempo total. Este efecto ya había sido anticipado como limitación conocida del diseño experimental: medir Java lanzando un proceso nuevo por archivo penaliza artificialmente el warm-up de la JVM en cada muestra.

## Python: overhead de intérprete, sin costo de arranque comparable al de Java

Python se ubicó en una posición intermedia: sin el costo fijo de arranque de una máquina virtual pesada como la JVM (el intérprete CPython arranca en pocos milisegundos), pero con un overhead por token/regla notablemente mayor que C++ debido a la interpretación de bytecode, el tipado dinámico y el conteo de referencias para la recolección de basura. El resultado es una curva claramente creciente y aproximadamente lineal, desde 21.1 ms (10 servicios) hasta 388.9 ms (500 servicios): a diferencia de Java, en Python sí es visible con claridad el efecto del tamaño de la entrada sobre el tiempo de ejecución, porque no hay un costo fijo de arranque que lo enmascare.

## Comparación conjunta

Para archivos pequeños (10–20 servicios), el orden observado es C++  $\ll$  Python  $\ll$  Java, con Java varias veces más lento que Python debido casi enteramente al arranque de la JVM. Para el archivo más grande (500 servicios), el orden de magnitud se mantiene (C++  $\approx$  24 ms, Python  $\approx$  389 ms, Java  $\approx$  557 ms), pero la diferencia relativa entre Java y Python se reduce de forma notable: el trabajo real de parsing en Python, al crecer linealmente con el tamaño, empieza a acercarse al costo fijo (y ya casi constante) de Java. De mantenerse esta tendencia para archivos aún mayores, sería razonable esperar que Python superara a Java en tiempo total, precisamente porque el costo de Java está dominado por un término prácticamente constante mientras que el de Python crece con el tamaño de la entrada.

En conjunto, el experimento evidencia el punto central que plantea el enunciado del proyecto: usando exactamente la misma gramática y el mismo metacompilador, la elección de lenguaje/runtime de implementación tiene un impacto medible y sistemático sobre los tiempos de ejecución, aunque en el caso de Java ese impacto está condicionado adicionalmente por la metodología de medición (un proceso nuevo por archivo), un efecto que conviene tener presente al interpretar la comparación.

## REFERENCIAS BIBLIOGRÁFICAS

- Parr, T. (2013). *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf. (Referencia oficial en [The Pragmatic Bookshelf](#)).
- Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707-710. (Abstract disponible en [IEEE Xplore](#)).
- Python Software Foundation. (2026). *difflib — Helpers for computing deltas — Python Documentation*. (Disponible en la [Documentación Oficial de Python](#)).
- Docker Inc. (2026). *Overview of Docker Compose*. (Disponible en la [Documentación Oficial de Docker](#)).